

Connecting the CareerHub UI to the real C# API

Swapping the Next.js mock route for the live ASP.NET Core + PostgreSQL backend.

Generated 22 June 2026 · Frontend branch `CAREERHUB-UI-1.3` · API branch `assignment-3.2`

What this achieves. The UI was reading 7 hard-coded jobs from an in-app mock route (`/api/jobs`). After these steps it reads **491 real listings** from PostgreSQL via the C# API at `http://localhost:5080` , with no changes to any React component — only the network layer and a type definition changed.

0 · Where the projects actually live

The paths differed from what was assumed. The real, working copies on this machine are:

Project	Path	Branch
Frontend (Next.js 15 / React 19)	<code>C:\Users\Xolisa\careerhub-1.3</code>	<code>CAREERHUB-UI-1.3</code>
Backend (ASP.NET Core / EF Core)	<code>C:\Users\Xolisa\assignment-3.2</code>	<code>assignment-3.2</code>

The folder `C:\Users\Xolisa\careerhub-api` is a **stray duplicate clone** of the same GitHub repo sitting on the UI branch — it is *not* the API and was ignored.

Key facts that differed from the original plan

Assumption	Reality
Envelope field is <code>items</code>	It is <code>data</code> (<code>PagedResponse.Data</code> → camelCase)
CORS may need <code>localhost:3000</code> added	NO CHANGE already present in <code>Program.cs</code>
Type fields roughly match	Big mismatch — list API returns far fewer fields
Stack is .NET 8	Project targets <code>net10.0</code> (runtime 10.0.8 installed)
API port	<code>5080</code> (from <code>launchSettings.json</code>)

1 Database — PostgreSQL via Docker DONE

Docker Desktop was installed but the daemon was not running. After starting it, the existing `docker-compose.yml` brings up Postgres 17 on host port **5544**, database **CareerHub24**, password `password123` — exactly matching `appsettings.Development.json`.

```
# From the API folder (only the db service - the API itself runs via dotnet run)
docker compose -f "C:\Users\Xolisa\assignment-3.2\docker-compose.yml" up -d db

# Confirm it is healthy
docker ps --filter "name=careerhub24-pg"
# careerhub24-pg   Up (healthy)   0.0.0.0:5544→5432/tcp
```

No Docker? Install PostgreSQL 17 for Windows, then during setup use port `5544` , set the `postgres` password to `password123` , and create a database named `CareerHub24` — the connection string then matches with zero edits.

2 API — run & verify DONE

The API auto-applies EF Core migrations and seeds sample South-African data on startup.

```
$env:ASPNETCORE_ENVIRONMENT = "Development"
dotnet run --project "C:\Users\Xolisa\assignment-3.2\CareerHub.Api.csproj" --launch-profile http
# → Now listening on: http://localhost:5080
```

Verify it returns the real envelope:

```
Invoke-RestMethod "http://localhost:5080/api/jobs?pageSize=2"
# envelope keys: data, page, pageSize, totalCount, totalPages, hasNextPage, hasPreviousPage, link
# totalCount: 491
# data[0]: { id, title, location, type:"FullTime", salaryMin, salaryMax,
#           status:"Active", createdAt, expiresAt, companyId, companyName, companyCity }
```

3 CORS NO CHANGE NEEDED

`Program.cs` already lists the SPA origin, so a browser at `localhost:3000` is allowed:

```
// Program.cs - already present
builder.Services.AddCors(options =>
    options.AddPolicy(CorsPolicy, policy => policy
        .WithOrigins("http://localhost:3000", "https://careerhub.example.com")
        .AllowAnyHeader().AllowAnyMethod().AllowCredentials()
        .WithExposedHeaders("X-Total-Count", "ETag", "api-supported-versions", "Retry-After")));
```

Confirmed live — a request carrying `Origin: http://localhost:3000` comes back with `Access-Control-Allow-Origin: http://localhost:3000` and `X-Total-Count: 491`.

4 Frontend — point at the API, unwrap the envelope, fix the types

DONE

4a • `.env.local` (created)

```
NEXT_PUBLIC_API_URL=http://localhost:5080
```

Restart `npm run dev` after creating this — Next.js inlines `NEXT_PUBLIC_*` at start time.

4b • Field mapping (API list item → UI `JobListing`)

The lean list projection (`JobListingResponse`) does **not** carry the long-form fields the UI model expects. We map what exists and default the rest; a detail page can later hydrate the heavy fields from `GET /api/jobs/{id}`.

UI field	Source from API
<code>company</code>	<code>companyName</code>
<code>employmentType</code>	<code>type</code> (enum name; <code>Learnership</code> added to the union)
<code>postedAt</code>	<code>createdAt</code>
<code>closingDate</code>	<code>expiresAt</code>
<code>isActive</code>	<code>status</code> $\equiv$ <code>"Active"</code>
<code>salaryMin</code> / <code>salaryMax</code>	same, with <code>?? 0</code> (API nullable $\rightarrow$ UI non-null)
<code>description</code> , <code>responsibilities</code> , <code>skills</code> , <code>minimumQualification</code> , <code>minimumExperienceYears</code> , <code>applicantCount</code>	NOT IN LIST API defaulted to <code>" "</code> / <code>[]</code> / <code>0</code>

4c · [src/types/index.ts](#)

Added an accurate `JobListingResponse` + `PagedResponse<T>` interface and the missing `"Learnership"` enum value. The original `JobListing` view-model is kept unchanged so all 8 consuming components keep compiling.

4d · [src/lib/api.ts](#) — the adapter

```
const payload = (await res.json()) as PagedResponse<JobListingResponse>;
return payload.data.map(toJobListing); // unwrap .data, adapt each record
```

5 Run both & verify end-to-end DONE

```
# Terminal 1 - API
dotnet run --project "C:\Users\Xolisa\assignment-3.2\CareerHub.Api.csproj" --launch-profile http

# Terminal 2 - UI
cd C:\Users\Xolisa\careerhub-1.3
npm run dev           #  $\rightarrow$  http://localhost:3000 (Environments: .env.local)
```

Verified: UI serves `200` ; API returns `491` listings; CORS header is correct. Open **`http://localhost:3000`** — you should now see hundreds of real listings from companies like *Capitec Bank*, not the 7 mock seeds.

6 Cleanup — disable the mock route DONE

Renamed so Next.js no longer treats it as a route handler:

```
src/app/api/jobs/route.ts → src/app/api/jobs/route.ts.bak
```

(With `NEXT_PUBLIC_API_URL` set, requests already go straight to `:5080` ; this just removes any chance of the mock silently taking over if the env var is ever unset.)

7 Completing the integration — detail endpoint & the apply flow

DONE

Connecting the list endpoint exposed two gaps the lean list projection left behind. Both are now closed.

7a • The `Learnership` enum value (compile fix)

The API's `JobType` enum has a value the UI union lacked: `Learnership` . Adding it to `EmploymentType` broke three *exhaustive* `Record<EmploymentType, ...>` maps (by design — they fail the build until the new key is supplied). All were updated:

- `src/lib/employmentType.ts` — label `"Learnership"`
- `src/components/JobStatusBadge.tsx` — a rose badge colour
- `src/lib/careerhub-store.ts` — store label
- `src/components/FilterBar.tsx` — a "Learnership" filter chip

7b • Detail endpoint — `fetchJobById`

Added `JobListingDetailResponse` (the API's detail DTO: it is the list shape plus `description` , `minimumRequirements` , `companyProvince` , `companyWebsite`) and a `fetchJobById(id)` in `src/lib/api.ts` that calls `GET /api/jobs/{id}` and hydrates the long-form fields the list could not supply.

```
GET http://localhost:5080/api/jobs/{id}
// → { ..., description: "...", minimumRequirements: "Matric.", companyProvince, companyWebsite }
```

7c · Live description in the SummaryPanel

When a job is selected on the home page, `SummaryPanel` now fetches its detail (same `["job", id]` cache the apply page uses) and shows the real **description** and **minimum requirements**. Sections the API genuinely has no data for (responsibilities, a structured skills list, experience years) render *only* when populated — no empty headers.

7d · The apply flow now works with real jobs BUG FIXED

The bug: `/apply/[jobId]` previously resolved jobs only from the mock `careerhub-store` (old seed ids). With real API GUIDs on the board, *every* "Apply" click would have shown *"Job not found."*

The page now loads the job from the API via TanStack Query, with a graceful fallback to the local store for recruiter-posted demo jobs:

```
const { data: apiJob, isPending, isError } = useQuery({
  queryKey: ["job", jobId],           // shares SummaryPanel's cache
  queryFn: () => fetchJobById(jobId),
  retry: false,                       // a 404 fails fast → local fallback
});
const localJob = getJobById(jobId); // recruiter-posted demo jobs
// → normalised into one ApplyJob shape; spinner while loading, "not found" only when both miss
```

Verified

- `npx tsc --noEmit` → exit 0 (all exhaustive maps satisfied).
- `GET /api/jobs/{id}` returns `description` + `minimumRequirements`.
- `/apply/{realGuid}` renders `200` (was the "not found" path before).

8 Full schema alignment — backend now matches the frontend

DONE

Four fields the UI model expected had no home anywhere in the API. Rather than leave them as permanent empty defaults, they were **added to the C# side** so the contract is honest end-

to-end.

Field	Backend change
<code>responsibilities</code>	New <code>List<string></code> → PostgreSQL <code>text[]</code> column
<code>skills</code>	New <code>List<string></code> → PostgreSQL <code>text[]</code> column
<code>minimumExperienceYears</code>	New <code>int</code> column (+ <code>≥ 0</code> check constraint)
<code>applicantCount</code>	Derived — server-side <code>COUNT</code> over the Applications navigation (not stored)

Files changed on the API (assignment-3.2)

- `Models/JobListing.cs` — three new properties
- `Data/CareerHubDbContext.cs` — `text[]` column mapping, SQL defaults (so existing rows backfill), experience check constraint
- `DTOs/Dtos.cs` — appended the fields to `JobListingResponse` & `JobListingDetailResponse` ; added optional inputs to the create/update requests
- `Repositories/JobListingRepository.cs` — all three projections + `PatchAsync`
- `Services/JobService.cs` — `CreateAsync` persists + validates the new fields
- `Data/SeedData.cs` — generators populate every seeded listing with realistic values

Migration & reseed

```
dotnet ef migrations add AddJobListingStructuredFields --project CareerHub.Api.csproj
# Wipe the dev volume so the updated seed regenerates listings WITH the new fields:
docker compose down -v && docker compose up -d db
dotnet run --project CareerHub.Api.csproj # auto-migrates + reseeds on startup
```

Gotcha: run with a build, *not* `--no-build`, right after creating a migration. `--no-build` runs the previously-compiled assembly, which lacks the just-generated migration + model snapshot, and EF Core 10 then throws `PendingModelChangesWarning` at startup.

Frontend follow-up

`src/types/index.ts` and `toJobListing` in `src/lib/api.ts` now read the real values straight off the list response (no more empty defaults). Only `description` and `minimumQualification` are still hydrated from the detail endpoint, since they are the heavy text fields the list projection omits.

Verified: `npx tsc --noEmit` → exit 0; API build → 0 errors; `GET /api/jobs` and `/api/jobs/{id}` both return populated `responsibilities`, `skills`, `minimumExperienceYears` and `applicantCount`; UI serves 200 with CORS intact.

Daily startup checklist

1. Start Docker Desktop → `docker compose ... up -d db` (or just confirm `careerhub24-pg` is healthy).
2. `dotnet run` in [assignment-3.2](#) → wait for *Now listening on http://localhost:5080*.
3. `npm run dev` in [careerhub-1.3](#) → open `http://localhost:3000`.

CareerHub integration walkthrough · Files touched: `.env.local` (new), `src/lib/api.ts`, `src/types/index.ts`, `src/app/api/jobs/route.ts` → `.bak`, `src/lib/employmentType.ts`, `src/lib/careerhub-store.ts`, `src/components/JobStatusBadge.tsx`, `src/components/FilterBar.tsx`, `src/components/SummaryPanel.tsx`, `src/app/apply/[jobId]/page.tsx`.